Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2026-27

Class:	**TE-AI&DS**	**Semester:**	**V**
Course Code:	**2015111**	**Course Name:**	**Statistics for Machine Learning and Data Science Lab**

Name of Student:	**Divya S. Konar**
Roll No. :	**20**
Experiment No.:	**9**
Title of the Experiment:	**Evaluate and Interpret Machine Learning Models using Validation and Explainability Technique**
Date of Performance:	**23/09/26**
Date of Submission:	**30/09/26**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Dr. Tatwadarshi P.N.

Signature :

Date :

Aim: To apply model evaluation, Bayesian inference concepts, and explainable AI techniques to interpret and understand machine learning predictions on the Pima Indians Diabetes Dataset.

Objectives

After completing this experiment, the student will be able to:

1. Apply model evaluation and Bayesian concepts to analyze the reliability of machine learning predictions.
2. Apply explainable AI techniques to interpret model predictions and identify important features influencing the results.

Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Matplotlib
- Scikit-learn
- SHAP
- LIME

Dataset Details

Dataset: Pima Indians Diabetes Dataset

The dataset contains diagnostic information for **768 female patients**.

The target variable is: Outcome = 0: Non-diabetic and Outcome = 1: Diabetic

Procedure

1. Load and preprocess the Pima Indians Diabetes Dataset.
2. Divide the dataset into training and testing sets.
3. Train a suitable classification model.
4. Evaluate the model using k-fold cross-validation.
5. Calculate suitable performance measures such as accuracy and F1-score.
6. Calculate the Brier Score and generate a calibration curve.
7. Explain selected model predictions using SHAP or LIME.
8. Identify the important features influencing the predictions.
9. Interpret and document the model evaluation and explainability results.

Description of the Experiment

A machine learning model should not be evaluated only on its performance on a single test split. Cross-validation provides a more reliable estimate of model performance.

The experiment follows: Dataset, Train Model, Cross-Validation, Performance Evaluation, Calibration Analysis, Explainable AI and Interpretation

The experiment combines three aspects:

- **Model Evaluation:** How well does the model perform?
- **Calibration:** How reliable are the predicted probabilities?
- **Explainability:** Why did the model make a particular prediction?

Detailed Description of Statistical Methods

k-Fold Cross-Validation

The dataset is divided into *k*subsets.

The model is trained on $k - 1$subsets and tested on the remaining subset.

This process is repeated *k*times.

The final performance is calculated using the average performance across all folds.

Bias-Variance Trade-Off

A model should balance:

- **Bias:** Error due to an overly simple model.
- **Variance:** Error due to a model being too sensitive to the training data.

A good model attempts to achieve a suitable balance between bias and variance.

Calibration Curve

A calibration curve compares:

$$Predicted\ Probability$$

with:

$$Observed\ Frequency$$

A well-calibrated model produces probabilities that closely match the actual frequency of the outcome.

Brier Score

The Brier Score measures the accuracy of probabilistic predictions:

$$BS = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2$$

where:

- p_i = predicted probability
- y_i = actual outcome

A lower Brier Score generally indicates better probabilistic predictions.

SHAP

SHAP values explain the contribution of individual features to a model prediction.

They help answer:

Which features contributed to increasing or decreasing the predicted probability?

LIME

LIME explains an individual prediction by creating small variations around the observation and approximating the model locally using an interpretable model.

Output :

[1] ✓ 2s

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    brier_score_loss,
    classification_report
)
from sklearn.calibration import calibration_curve

# Load dataset
url = "https://raw.githubusercontent.com/npradaschnor/Pima-Indians-Diabetes-Dataset/master/diabetes.csv"

df = pd.read_csv(url)

print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
```

```
Dataset Shape: (768, 9)

First 5 rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

[2]
```
# Columns where 0 is considered an invalid/missing value
columns_to_replace = [
    'Glucose',
    'BloodPressure',
    'SkinThickness',
    'Insulin',
    'BMI'
]

# Replace 0 with NaN
df[columns_to_replace] = df[columns_to_replace].replace(0, np.nan)

# Fill missing values with median
for column in columns_to_replace:
    df[column] = df[column].fillna(df[column].median())

print("Missing values after preprocessing:")
print(df.isnull().sum())
```

```
Missing values after preprocessing:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

[3]
```
# Features
X = df.drop('Outcome', axis=1)

# Target
y = df['Outcome']

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature shape: (768, 8)
Target shape: (768,)
```

[5]
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 614
Testing samples: 154
```

[6]
```
# Create Logistic Regression pipeline
model = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(random_state=42))
])

# Train the model
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

[7]
```
# Predict class labels
y_pred = model.predict(X_test)

# Predict probabilities
y_prob = model.predict_proba(X_test)[:, 1]

print("First 10 predicted classes:")
print(y_pred[:10])

print("\nFirst 10 predicted probabilities:")
print(y_prob[:10])
```

```
First 10 predicted classes:
[1 0 0 0 0 0 0 1 0 1]

First 10 predicted probabilities:
[0.60971073 0.11835576 0.29346313 0.2508367  0.03320667 0.16852385
 0.4881643  0.92367511 0.08143405 0.8385896 ]
```

[8]
```
from sklearn.model_selection import StratifiedKFold, cross_val_score

# 5-Fold Stratified Cross-Validation
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

# Accuracy
cv_accuracy = cross_val_score(
    model,
    X,
    y,
    cv=cv,
    scoring='accuracy'
)

# F1-score
cv_f1 = cross_val_score(
    model,
    X,
    y,
    cv=cv,
    scoring='f1'
)

print("Accuracy for each fold:")
print(cv_accuracy)

print("\nMean Accuracy:", cv_accuracy.mean())
```

[8]
```
    y,
    cv=cv,
    scoring='f1'
)

print("Accuracy for each fold:")
print(cv_accuracy)

print("\nMean Accuracy:", cv_accuracy.mean())

print("\nF1-score for each fold:")
print(cv_f1)

print("\nMean F1-score:", cv_f1.mean())
```

```
Accuracy for each fold:
[0.77272727 0.7987013  0.77922078 0.75163399 0.76470588]

Mean Accuracy: 0.7733978439860792

F1-score for each fold:
[0.63917526 0.65168539 0.63829787 0.59574468 0.66037736]

Mean F1-score: 0.6370561125344882
```

[9]
```
test_accuracy = accuracy_score(y_test, y_pred)
test_f1 = f1_score(y_test, y_pred)

print("Test Accuracy:", test_accuracy)
print("Test F1-score:", test_f1)
```

```
Test Accuracy: 0.7077922077922078
Test F1-score: 0.5454545454545454
```

[10] ✓ 0s

```
from sklearn.metrics import brier_score_loss

# Calculate Brier Score
brier_score = brier_score_loss(y_test, y_prob)

print("Brier Score:", brier_score)
```

```
Brier Score: 0.17541713693275893
```

[11] ✓ 0s

```
from sklearn.calibration import calibration_curve

# Calculate calibration values
prob_true, prob_pred = calibration_curve(
    y_test,
    y_prob,
    n_bins=10,
    strategy='uniform'
)

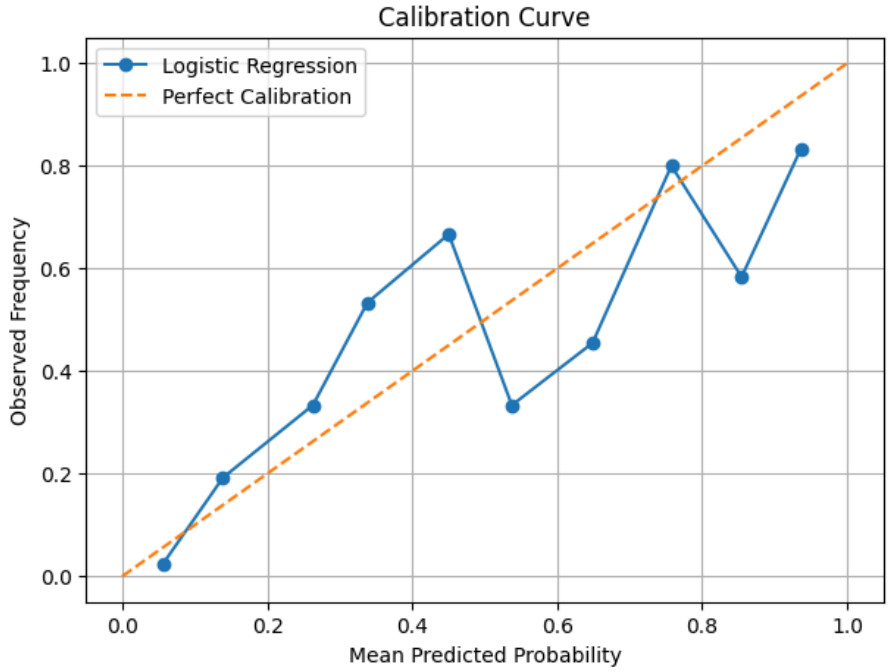
# Plot calibration curve
plt.figure(figsize=(7, 5))

plt.plot(
    prob_pred,
    prob_true,
    marker='o',
    label='Logistic Regression'
)

# Perfect calibration line
plt.plot(
    [0, 1],
```

```
# Perfect calibration line
plt.plot(
    [0, 1],
    [0, 1],
    linestyle='--',
    label='Perfect Calibration'
)

plt.xlabel("Mean Predicted Probability")
plt.ylabel("Observed Frequency")
plt.title("Calibration Curve")
plt.legend()
plt.grid()
plt.show()
```



[12] ✓ 11s
```
!pip install shap -q
```

[13] ✓ 5s
```
import shap
```

[14] ✓ 0s
```
# Get the trained scaler
scaler = model.named_steps['scaler']

# Transform test data
X_test_scaled = scaler.transform(X_test)

# Convert to DataFrame
X_test_scaled_df = pd.DataFrame(
    X_test_scaled,
    columns=X_test.columns
)

print(X_test_scaled_df.head())
```

```
   Pregnancies   Glucose  BloodPressure  SkinThickness   Insulin       BMI  \
0     0.960541  1.245157      -0.663676      -0.004766 -0.161437 -0.740145
1     1.866489 -1.790266       2.760669      -0.680091 -1.127122  0.447785
2    -0.549372  0.010974       0.314708      -0.229874  0.791541  0.506448
3     0.960541 -0.255876      -0.663676      -0.004766 -0.161437 -0.740145
4    -0.549372 -1.590128      -0.011420      -0.004766 -0.161437 -0.021521

   DiabetesPedigreeFunction       Age
0                 -0.555791  0.561034
1                 -0.583061  1.153060
2                  0.016882 -0.623019
3                  0.771356  0.053583
4                 -1.137554 -0.961320
```

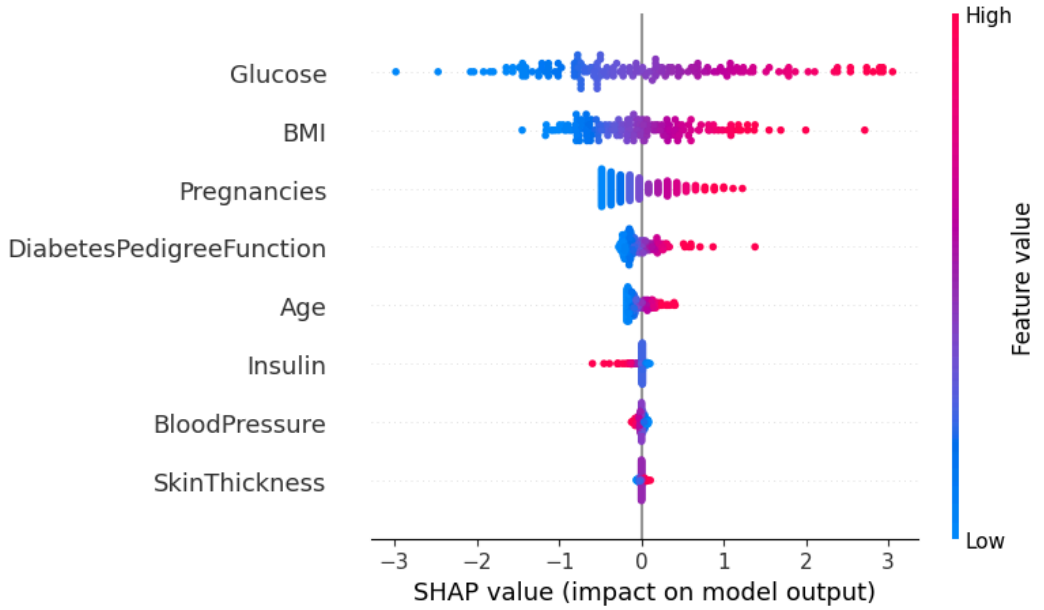
```
[15]
✓ 0s
# Create SHAP explainer
explainer = shap.LinearExplainer(
    model.named_steps['classifier'],
    scaler.transform(X_train)
)

# Calculate SHAP values
shap_values = explainer.shap_values(X_test_scaled)

print("SHAP values calculated successfully!")
```

```
WARNING:shap:Background dataset has 614 samples but max_samples=100. Subsampling to 100 samples for SHAP value computation. To use all samples, set max_samples=614 when initializing the masker.
SHAP values calculated successfully!
```

```
[16]
✓ 1s
shap.summary_plot(
    shap_values,
    X_test_scaled_df,
    feature_names=X_test.columns
)
```




```
[17]
✓ 0s
# Calculate mean absolute SHAP values
shap_importance = np.abs(shap_values).mean(axis=0)

feature_importance = pd.DataFrame({
    'Feature': X_test.columns,
    'Mean Absolute SHAP Value': shap_importance
})

# Sort by importance
feature_importance = feature_importance.sort_values(
    by='Mean Absolute SHAP Value',
    ascending=False
)

print(feature_importance)
```

```
                    Feature  Mean Absolute SHAP Value
1                   Glucose                  1.015550
5                       BMI                  0.566419
0               Pregnancies                  0.341287
6  DiabetesPedigreeFunction                  0.173990
7                       Age                  0.126332
4                   Insulin                  0.046624
2             BloodPressure                  0.031152
3             SkinThickness                  0.017932
```

[18]
```
# Select the first patient from the test set
patient_index = 0

patient_data = X_test_scaled_df.iloc[[patient_index]]

print("Patient features:")
print(X_test.iloc[patient_index])

print("\nActual Outcome:", y_test.iloc[patient_index])
print("Predicted Outcome:", y_pred[patient_index])
print("Predicted Diabetes Probability:", y_prob[patient_index])
```

```
Patient features:
Pregnancies                 7.000
Glucose                   159.000
BloodPressure              64.000
SkinThickness              29.000
Insulin                   125.000
BMI                        27.400
DiabetesPedigreeFunction    0.294
Age                        40.000
Name: 44, dtype: float64

Actual Outcome: 0
Predicted Outcome: 1
Predicted Diabetes Probability: 0.6097107330034057
```

[19]
```
# Get SHAP values for the selected patient
patient_shap = shap_values[patient_index]

# Create a DataFrame
patient_explanation = pd.DataFrame({
    'Feature': X_test.columns,
    'Feature Value': X_test.iloc[patient_index].values,
    'SHAP Value': patient_shap
})

# Sort by absolute SHAP value
patient_explanation['Absolute SHAP'] = np.abs(
    patient_explanation['SHAP Value']
)

patient_explanation = patient_explanation.sort_values(
    by='Absolute SHAP',
    ascending=False
)

print(patient_explanation)
```

```
                    Feature  Feature Value  SHAP Value  Absolute SHAP
1                   Glucose        159.000    1.553313       1.553313
5                       BMI         27.400   -0.518578       0.518578
0               Pregnancies          7.000    0.315778       0.315778
6  DiabetesPedigreeFunction          0.294   -0.123011       0.123011
7                       Age         40.000    0.060000       0.060000
2             BloodPressure         64.000    0.030395       0.030395
4                   Insulin        125.000    0.011466       0.011466
3             SkinThickness         29.000    0.000921       0.000921
```

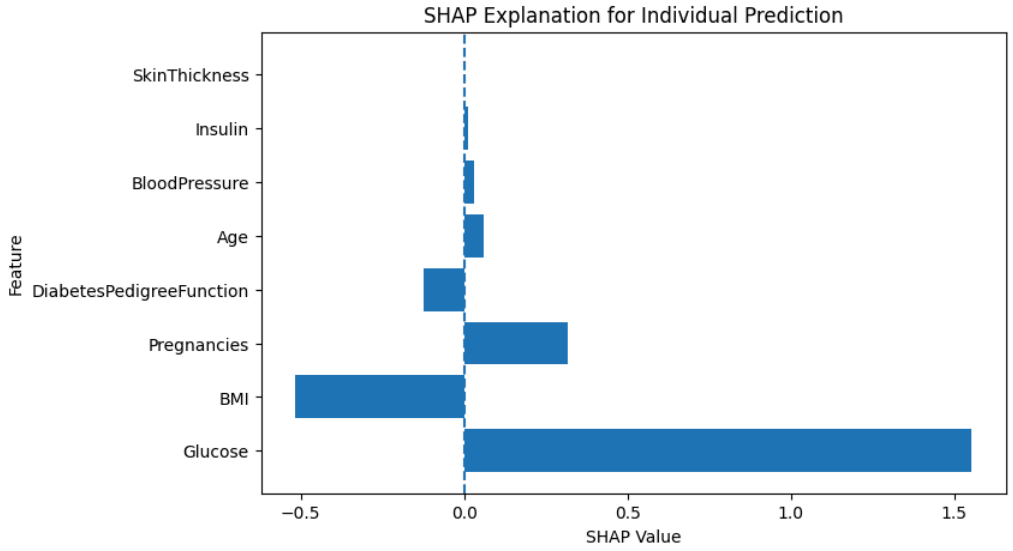
[20] ✓ 0s

```
plt.figure(figsize=(8, 5))

plt.barh(
    patient_explanation['Feature'],
    patient_explanation['SHAP Value']
)

plt.xlabel("SHAP Value")
plt.ylabel("Feature")
plt.title("SHAP Explanation for Individual Prediction")
plt.axvline(x=0, linestyle='--')

plt.show()
```



[21] ✓ 0s

```
print("========== MODEL EVALUATION RESULTS ==========")

print("Mean Cross-Validation Accuracy:",
      round(cv_accuracy.mean(), 4))

print("Mean Cross-Validation F1-score:",
      round(cv_f1.mean(), 4))

print("Test Accuracy:",
      round(test_accuracy, 4))

print("Test F1-score:",
      round(test_f1, 4))

print("Brier Score:",
      round(brier_score, 4))
```

```
========== MODEL EVALUATION RESULTS ==========
Mean Cross-Validation Accuracy: 0.7734
Mean Cross-Validation F1-score: 0.6371
Test Accuracy: 0.7078
Test F1-score: 0.5455
Brier Score: 0.1754
```

```
[22]
✓ 0s
print("========== SHAP FEATURE IMPORTANCE ==========")

print(
    feature_importance[
        ['Feature', 'Mean Absolute SHAP Value']
    ].to_string(index=False)
)
```

```
========== SHAP FEATURE IMPORTANCE ==========
                  Feature  Mean Absolute SHAP Value
                  Glucose                  1.015550
                      BMI                  0.566419
              Pregnancies                  0.341287
DiabetesPedigreeFunction                   0.173990
                      Age                  0.126332
                  Insulin                  0.046624
            BloodPressure                  0.031152
            SkinThickness                  0.017932
```

```
[23]
✓ 0s
print("===== FINAL VALUES =====")

print("Accuracy Mean:", cv_accuracy.mean())
print("Accuracy SD:", cv_accuracy.std())

print("F1 Mean:", cv_f1.mean())
print("F1 SD:", cv_f1.std())

print("Brier Score:", brier_score)

print("\nSHAP Feature Importance:")
print(feature_importance)

print("\nIndividual Prediction:")
print("Actual:", y_test.iloc[0])
print("Predicted:", y_pred[0])
print("Probability:", y_prob[0])
print("\nIndividual SHAP:")
print(patient_explanation)
```

```
===== FINAL VALUES =====
Accuracy Mean: 0.7733978439860792
Accuracy SD: 0.015645042231695692
F1 Mean: 0.6370561125344882
F1 SD: 0.02222864592799808
Brier Score: 0.17541713693275893

SHAP Feature Importance:
                  Feature  Mean Absolute SHAP Value
1                 Glucose                  1.015550
5                     BMI                  0.566419
0             Pregnancies                  0.341287
6  DiabetesPedigreeFunction                  0.173990
7                     Age                  0.126332
4                 Insulin                  0.046624
2           BloodPressure                  0.031152
3           SkinThickness                  0.017932

Individual Prediction:
Actual: 0
Predicted: 1
Probability: 0.6097107330034057

Individual SHAP:
                  Feature  Feature Value  SHAP Value  Absolute SHAP
1                 Glucose        159.000    1.553313       1.553313
5                     BMI         27.400   -0.518578       0.518578
0             Pregnancies          7.000    0.315778       0.315778
6  DiabetesPedigreeFunction          0.294   -0.123011       0.123011
7                     Age         40.000    0.060000       0.060000
2           BloodPressure         64.000    0.030395       0.030395
4                 Insulin        125.000    0.011466       0.011466
3           SkinThickness         29.000    0.000921       0.000921
```

Conclusion

The classification model was evaluated using cross-validation and calibration analysis to understand its performance and reliability. Explainable AI techniques such as SHAP or LIME were applied to identify the features influencing individual predictions. The experiment demonstrated that model performance, probability reliability, and interpretability are important aspects of evaluating machine learning systems.

Questions to be Answered by Students

1. What was the mean and standard deviation of the model performance across the cross-validation folds?

ANS : The mean cross-validation accuracy was 0.73398 with a standard deviation of 0.01565. The mean cross-validation F1-score was 0.63761 with a standard deviation of 0.02223. The relatively small standard deviations indicate that the model performance was fairly consistent across the five folds.

2. What was the obtained Brier Score, and what does it indicate about the reliability of the predicted probabilities?

ANS : The obtained Brier Score was 0.17542. A lower Brier Score indicates more accurate probabilistic predictions. The obtained value indicates that the model's predicted probabilities had a reasonable level of reliability, although they were not perfectly accurate.

3. Was the model well calibrated based on the calibration curve? Justify your answer.

ANS : The Logistic Regression model was reasonably calibrated, but not perfectly calibrated. The calibration curve follows the diagonal Perfect Calibration line in several regions, indicating that the predicted probabilities are reasonably close to the observed frequencies. However, there are noticeable deviations from the diagonal at some probability ranges, especially around 0.4–0.6 and 0.8–0.9. Therefore, the model provides reasonably reliable probabilities but shows some calibration error.

4. Which features were identified as the most influential using SHAP or LIME?

ANS : Glucose — 1.015550

1. BMI — 0.566419
2. Pregnancies — 0.341287
3. DiabetesPedigreeFunction — 0.173990

4. Age — 0.126332
5. Insulin — 0.046624
6. BloodPressure — 0.031152
7. SkinThickness — 0.017932

SHAP analysis identified Glucose as the most influential feature, followed by BMI and Pregnancies. DiabetesPedigreeFunction and Age also had noticeable influence on the model predictions. Glucose had the highest mean absolute SHAP value of 1.01555, indicating the greatest overall contribution among the features.

5. Explain one individual prediction using the obtained explainability results.

ANS : For the selected patient, the actual outcome was 0 (Non-diabetic), but the Logistic Regression model predicted 1 (Diabetic) with a probability of 60.97%. The SHAP analysis showed that Glucose (159) had the strongest positive contribution with a SHAP value of +1.55331, pushing the prediction toward the diabetic class. Pregnancies (7) also increased the prediction with a SHAP value of +0.31578. On the other hand, BMI (27.4) contributed negatively with a SHAP value of −0.51858, which pushed the prediction toward the non-diabetic class. Overall, the positive contributions, particularly from Glucose, resulted in the model predicting the patient as diabetic despite the actual outcome being non-diabetic.